Vektör Çarpımı ve Döndürme Simülasyonu - Kod Açıklamaları

Bu doküman, GitHub'da paylaşılan `cross product and vector rotation.py` kodunun matematiksel arka planını ve hesaplama adımlarını açıklamaktadır. Kod, VPython kütüphanesini kullanarak üç boyutlu uzayda iki vektörün (A ve B) vektörel çarpımını hesaplamakta ve A vektörünün B vektörü üzerine dönmesini (rotasyonunu) simüle etmektedir.

1. Vektörlerin Tanımlanması ve Vektörel Çarpım (Cross Product)

Kodun başlangıcında kullanıcıdan A ve B vektörlerinin x, y ve z bileşenleri alınır. Bu bileşenler VPython'ın `vector` sınıfı kullanılarak üç boyutlu vektör objelerine dönüştürülür.

Hesaplama:
İki vektörün vektörel çarpımı **A × B**, VPython kütüphanesindeki cross(a, b) fonksiyonu ile hesaplanır.

Sonuç olarak elde edilen vektör (kodda crossp), A ve B vektörlerinin oluşturduğu düzleme dik (ortogonal) olan ve dönme eksenini temsil eden vektördür.

2. Vektörler Arasındaki Açının Bulunması

A vektörünün B vektörü üzerine dönebilmesi için öncelikle aralarındaki açının bilinmesi gerekir.

Hesaplama:
Kod içerisinde a.diff_angle(b) metodu kullanılarak iki vektör arasındaki açı (radyan cinsinden) hesaplanır (kodda `ang` değişkeni).

Matematiksel olarak bu hesaplama skaler çarpım (dot product) kullanılarak yapılır. Kodda, vektörlerin büyüklüklerinin sıfır olma durumu veya birbirlerine paralel/anti-paralel olma durumu (vektörel çarpımın büyüklüğünün sıfıra çok yakın olması durumu, mag(crossp) < 1e-6) kontrol edilerek hata alınması önlenmiştir.

3. Dönme Ekseni (Rotation Axis) Birim Vektörünün Bulunması

Vektörü döndürmek için ekseni belirlememiz gerekir. Bu eksen, A ve B vektörlerine dik olan A x B vektörünün doğrultusundadır.

Hesaplama:

Dönme eksenini temsil eden birim vektör u (ux, uy, uz), vektörel çarpım sonucunun kendi büyüklüğüne (normuna) bölünmesiyle elde edilir.

Kodda bu işlem şu satırlarda gerçekleşir:

```python
rotaks = crossp / mag(crossp)
ux = rotaks.x
uy = rotaks.y
uz = rotaks.z
```

4. 3 Boyutlu Uzayda Keyfi Bir Eksen Etrafında Döndürme Matrisi (Rodrigues Rotation Formula)

Kodun en temel matematiksel bileşeni olan dönme işlemi, Euler-Rodrigues formülü baz alınarak oluşturulan 3D Dönme Matrisi ile gerçekleştirilir. Kod içerisindeki Rot(x) fonksiyonu bu matrisi tanımlar. x parametresi, o anki dönme açısını (radyan cinsinden) temsil eder.

Hesaplama:

Bir vektörü, birim vektörü u = (ux, uy, uz) olan bir eksen etrafında x açısı kadar döndüren 3×3'lük matris şu şekildedir:

Satır 1: [ux^2(1 - cos(x)) + cos(x), ux*uy(1 - cos(x)) - uz*sin(x), ux*uz(1 - cos(x)) + uy*sin(x)]

Satır 2: [ux*uy(1 - cos(x)) + uz*sin(x), uy^2(1 - cos(x)) + cos(x), uy*uz(1 - cos(x)) - ux*sin(x)]

Satır 3: [ux*uz(1 - cos(x)) - uy*sin(x), uy*uz(1 - cos(x)) + ux*sin(x), uz^2(1 - cos(x)) + cos(x)]

Kodda bu matris, NumPy kütüphanesi yardımıyla np.array olarak döndürülür.

5. Simülasyon Döngüsü ve Matris Çarpımı

Animasyon sırasında A vektörü adım adım küçük açılarla döndürülerek simülasyon oluşturulur.

Hesaplama:

Her zaman adımında (dt), mevcut teta açısı için dönme matrisi hesaplanır. A vektörü (NumPy dizisi formatında a_np), bu dönme matrisi Rot(teta) ile çarpılır.
Bu matris-vektör çarpımı kodda NumPy'ın .dot() metodu ile yapılır:

```python
xrot_np = Rot(teta).dot(a_np)
```

Elde edilen yeni vektör (xrot_np), VPython sahnesindeki hareketli kırmızı ok objesinin (vecrot.axis) güncel pozisyonu olarak ayarlanır. Bu işlem, teta açısı toplam ang değerine ulaşana kadar tekrarlanır ve vektörün düzlem içindeki pürüzsüz rotasyon animasyonu gerçekleşir.